İşletim Sistemleri
proses Senkronizasyonu 3

Dr. Öğr. Üyesi Ertan Bütün

Bu dersin içeriği hazırlanırken Operating System Concepts (Silberschatz, Galvin and Gagne) kitabı ve
Prof. Dr. M. Ali Akcayol'un (Gazi Üniversitesi Bilgisayar Mühendisliği Bölümü) ders sunumlarından faydalanılmıştır.

https://codex.cs.yale.edu/avi/os-book/OS9/slide-dir/index.html
http://w3.gazi.edu.tr/~akcayol/BMOS.htm

Konular

- Process Synchronization
- The Critical-Section Problem
- Peterson’s Solution
- Synchronization Hardware
- Mutex Locks
- Semaphores
- Classical Problems of Synchronization
- Monitors
- Alternative Approaches

Mutex Locks

- Donanım tabanlı critical section (CS) çözümleri karmaşıktır ve uygulama geliştirici tarafından erişilemez.
- İşletim sistemi tasarımcıları CS problemi için yüksek-seviye yazılım araçları geliştirmişlerdir.
- Bu yazılım araçlarının en basiti mutex (mut*ual* ex*clusion*) locks aracıdır.
- Bir proses CS'ye girmek için önce kilidi alır (acquire()); CS'den çıktığında kilidi serbest bırakır (release()).
- Lock durumunun uygun olup olmadığına karar vermek için mutex lock, boolean türünden **available** değişkeni kullanır.

Mutex Locks

- Aşağıda, acquire(), release() fonksiyonları ile mutex lock kullanılan **CS çözümü verilmiştir. acquire() ve release()** çağrıları **atomic yürütülmelidir**. Genellikle atomic donanım komutları ile gerçekleştirilir

```
acquire() {
    while (!available)
        ; /* busy wait */
    available = false;
}
```

```
release() {
    available = true;
}
```

```
do {
    acquire lock
        critical section
    release lock
        remainder section
} while (true);
```

- Mutext locks'ın **temel dezavantajı** meşgul bekleme **(busy waiting)** gerektirmesidir. Bir proses CS'de iken , CS'ye girmeye çalışan diğer herhangi bir proses, acquire() çağrısında while döngüsünde sürekli bekler. **busy waiting** , başka bir prosesin verimli bir şekilde kullanabileceği CPU'yu boşta bekletir.*

Mutex Locks

- Mutex locks’a aynı zamanda spinlock da denir, çünkü proses kilidin kullanılabilir hale gelmesini beklerken “spin” eder.
- Spinlock'ların bir avantajı: bir prosesin bir kilidi beklemesi gerektiğinde context switch’e gerek kalmamasıdır, bir context switch önemli ölçüde zaman alabilir.
- Spinlock’lar, meşgul bekleme problemi açısından tek çekirdekli sistemlerde CPU’nun birçok proses arasında paylaşıldığı multiprogramming sisteme uygun değildir.
  - Çünkü meşgul bekleme durumunda olan proses CPU’ya geçtiğinde döngüde sadece bekleyecek, tek çekirdekli sistemde bu süre boyunca bir iş yapılmamış olacak.

Mutex Locks

- Lock kısa bir süre için tutulacaksa, çok çekirdekli (multicore) sistemlerde tercih edilen kilitleme mekanizması spinlock'lardır.
- Bir lock kısa bir süre için tutulacaksa, bir thread bir işlemci çekirdeğinde spin edebilir yani "dönebilir" , diğer threadler ise başka işlemci çekirdeklerinde çalışabilir ve böylece meşgul beklemede olan thread'in beklediği durum diğer threadlerin çalışmasıyla sağlanabilir.
- Modern multicore sistemler için spinlock'lar çoğu işletim sisteminde yaygın olarak kullanılmaktadır.

Konular

- Process Synchronization
- The Critical-Section Problem
- Peterson’s Solution
- Synchronization Hardware
- Mutex Locks
- Semaphores
- Monitors
- Alternative Approaches

Semaphores

- Mutex locks, genellikle en basit senkronizasyon araçları olarak kabul edilir.
- Semaforlar, mutex locks'ların işleyişine benzer, ancak aynı zamanda proses senkronizasyonu için daha gelişmiş yollar sunarlar.

Semaphores

- S semaforu bir tamsayıdır ve S'ye sadece wait() ve signal() **atomik** işlemleri tarafından erişilebilir.
- Literatürde **wait()** işlemi **P()** ile, **signal()** işlemi ise **V()** ile de gösterilir.
- **wait()** ve **signal()** işlemleri aşağıda verilmiştir.*

```
wait(S) {
    while (S <= 0)
        ; // busy wait
    S--;
}
```

```
signal(S) {
    S++;
}
```

- S üzerindeki **wait()** ve **signal()** işlemleri atomik (kesintisiz) bir şekilde gerçekleştirilir. Yani bir proses semafor değerini değiştirirken başka bir poses aynı anda semafor değerini değiştiremez.
- Ayrıca **wait()** içindeki S'nin negatif olup olmadığı testi de interrupt olmadan gerçekleştirilmelidir.

Semaphores

- İşletim sistemleri, counting semaphore ve binary semaphore kullanırlar.
- Counting semaphore değeri kısıtlı olmayan bir alan üzerinde değişebilir.
  - Belirli sayıdaki kaynağa erişimi denetlemek için kullanılır.
  - Kaynak sayısı ile başlatılır.
- Binary semaphore değeri 0 veya 1 olabilir.
  - Mutex locks gibi davranır.

Semaphores

- Semaforlar, **işlem bağımlılığı gibi farklı senkronizasyon problemlerinde de kullanılabilir.**
- Örneğin, P_2 prosesindeki S_2 komutu P_1 prosesindeki S_1 komutundan sonra çalışmak zorunda olsun. Bu durum P_1 ve P_2 nin ortak bir binary semafor (adı synch olsun) paylaşmasıyla nasıl gerçekleştirilebilir?
- Synch semaforu için wait(synch)/signal(synch) çağrıları P_1 ve P_2 proseslerinde nerede çağrılmalı? Synch semaforunun başlangıç değeri ne olmalıdır?

P_1	P_2
S_1 ;	S_2 ;

Semaphores

- **synch** semaforunun başlangıç değeri 0 yapılır.
- **P_1 ve P_2 içerisine eklenen komutlar şöyledir:***

P_1

```
S1;
signal(synch);
```

synch = 0

P_2

```
wait(synch);
S2;
```

synch artırılır.

synch > 0 oluncaya kadar bekler.

Semaphore Gerçekleştirimi

- Mutex locks gibi semafordaki wait() ve signal() için de meşgul bekleme (**busy waiting**) problemi söz konusudur.
- **Busy waiting problemi şöyle çözülebilir**:
  - Proses, semafor değeri pozitif olmadığında while döngüsünde busy waiting yapmak yerine kendini **bloke** edebilir.
  - **Bloke işlemi** ile proses, semafor ile ilişkili bir bekleme kuyruğuna alınır ve durumu **waiting state** olur.
  - Daha sonra CPU scheduler başka bir prosesi CPU'da yürütmek üzere seçer.
  - Bir S semaforunda bekleyen bloke edilmiş bir proses, başka bir proses bir signal() işlemi yürüttüğünde yeniden başlatılmalıdır. Processin durumunu **waiting state**'ten **ready state**'ye değiştiren **wakeup() işlemiyle** proses yeniden başlatılır.
  - proses daha sonra ready queue'ye yerleştirilir.

Semaphore Gerçekleştirimi

- Bu çözüm için semafor şöyle tanımlanır:

```
typedef struct {
     int value;
     struct process *list;
} semaphore;
```

- Her semafor bir tamsayı değeri ve **proses listesine** sahiptir.
- **Bir proses semaforu bekliyorsa proses listesine eklenir.**
- signal() işlemi ile proses listesinde bekleyen bir proses çıkarılır ve bu proses uyandırılır.

Semaphore Gerçekleştirimi

- Semafor için wait() ve signal() işlemleri aşağıdaki gibi tanımlanır:
- **block()** prosesi **beklemeye alır**, **wakeup()** ise çalışmaya **devam ettirir.**

```
wait(semaphore *S) {
        S->value--;
        if (S->value < 0) {
                add this process to S->list;
                block();
        }
}
```

Negatif değer, bekleyen proses sayısını gösterir.

```
signal(semaphore *S) {
        S->value++;
        if (S->value <= 0) {
                remove a process P from S->list;
                wakeup(P);
        }
}
```

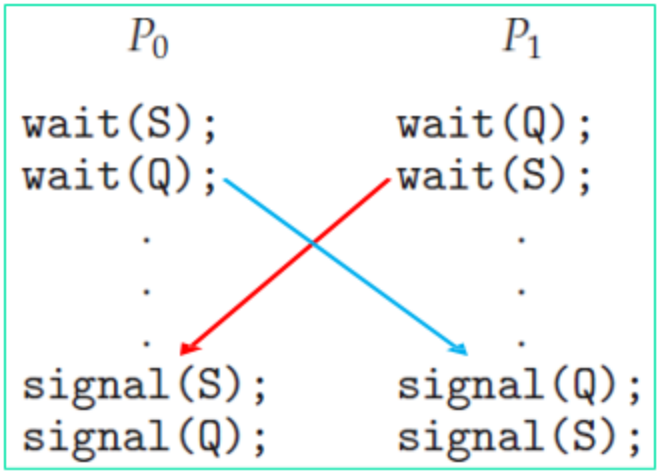
```
typedef struct {
    int value;
    struct process *list;
} semaphore;
```

Deadlock (Ölümcül Kilitlenme)

- Semaforların waiting queue ile gerçekleştirimi şöyle bir duruma neden olabilir:
  - İki ya da daha fazla proses devam edebilmek için birbirlerinden bir olayı bitirmesini bekliyorsa ve hiçbir proses devam edemiyor karşılıklı istedikleri olayın gerçekleşmesini bekliyorsa bu duruma **deadlock** denir.
  - **Deadlock'da** iki veya daha fazla prosesin sonsuza kadar birbirlerini beklemesi söz konusudur.

Deadlock (Ölümcül Kilitlenme)

- Örneğin P_0 ve P_1 prosesleri, **S** ve **Q** semaforlarına erişiyor olsun. **S** ve **Q** nun başlangıç değeri 1 olsun.
- P_0 wait(S) ve P_1 wait(Q) işlemlerini çalıştırsın. Bu durumda **S** ve **Q** nun değerleri 0 olacaktır. Bu noktandan sonra P_0 wait(Q)'yu çalıştırdığında ***P1*** ın signal(Q) yi çalıştırmasını bekleyecektir. Benzer şekilde P_1 wait(S)'yi çalıştırdığında ***P0*** ın signal(S)'yi çalıştırmasını bekleyecektir.
- Bu yüzden signal() işlemleri yürütülemeyecek P_0 ve P_1 deadlock olacaktır.
- Deadlock sorunuyla başa çıkmak için çeşitli mekanizmalar Deadlock bölümünde açıklanacaktır.



Starvation (Açlık)

- Deadlock'larla ilgili diğer bir sorun, processlerin semafor içinde sonsuza kadar beklediği bir durum olan **indefinite blocking** veya **starvation dır**.
- **Starvation** durumunda yürütülmeye hazır bir proses CPU'dan kaynak tahsisi etmesini bekler. Ancak, diğer processler sürekli olarak istenen kaynakları bloke ettiği için proses süresiz olarak bekleme durumunda olur.
- Örneğin, bir semaforla ilişkili processler listeden LIFO (son giren, ilk çıkar) kuyruk yapısına göre çıkarılırsa **starvation** meydana gelebilir.
- Bir **deadlock** durumunda, processlerin hiçbiri yürütme için ilerlemez, her proses başka bir proses tarafından edinilen kaynakları beklerken engellenir.
- **Starvation** durumunda ise daha yüksek önceliğe sahip processlerin kaynakları sürekli olarak elde etmesiyle, düşük öncelikli processlerin kaynak elde etmesinin önüne geçilir, düşük öncelikli processler belirsiz bir şekilde engellenir.

Priority Inversion

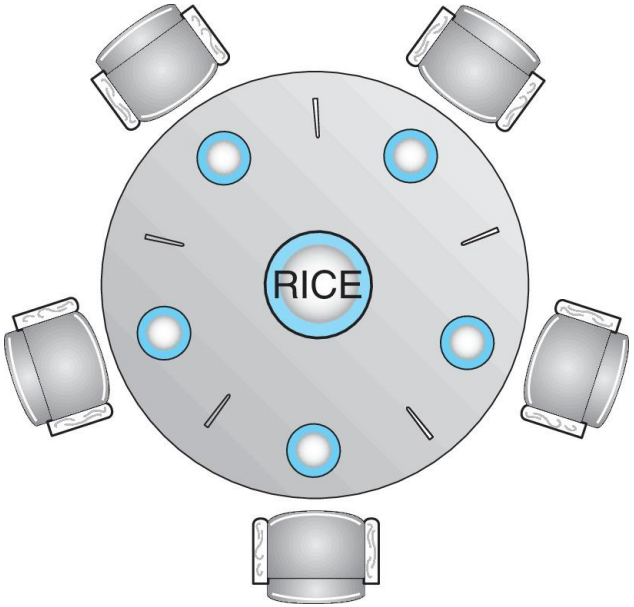
- Yüksek öncelikli bir prosesin, aynı anda düşük öncelikli bir proses tarafından erişilen kernel verilerini okuması veya değiştirmesi gerektiğinde, scheduling zorluğu ortaya çıkar.
- Örneğin öncelik sırası L < M < H olan prosesler olsun.
- Kernel verileri tipik olarak bir lock ile korunduğundan, daha yüksek öncelikli proses (H), daha düşük öncelikli prosesin (L) kaynağı bırakması için bir süre beklemek zorunda kalacaktır.
- Düşük öncelikli proses (L), daha yüksek önceliğe sahip başka bir proses (M) lehine önceliklendirilirse durum daha karmaşık hale gelir.
- Bu problem **priority inversion** olarak bilinir. **priority-inheritance protocol** bu problemin çözümü için uygulanan bir yöntemdir.
- Bu protokole göre, daha yüksek öncelikli bir processin ihtiyaç duyduğu kaynaklara erişen tüm prosesler, söz konusu kaynaklarla işi bitene kadar daha yüksek önceliği devralır. İşlerini bitirdiklerinde öncelikleri orijinal değerlerine geri döner. Böylece daha düşük öncelikli bir proses araya girip kaynağı alamaz.

Classical Problems of Synchronization

- Bu bölümde örnek olarak birkaç senkronizasyon problemi verilmiştir. Bu problemler, neredeyse önerilen her yeni senkronizasyon yöntemini test etmek için kullanılan problemlerdir:
  - Dining-Philosophers Problem
  - Bounded-Buffer Problem (also called Producer Consumer)
  - Readers and Writers Problem
- Bu bölümde bu problemlerin senkronizasyon çözümü için **semaforlar** kullanılmıştır.

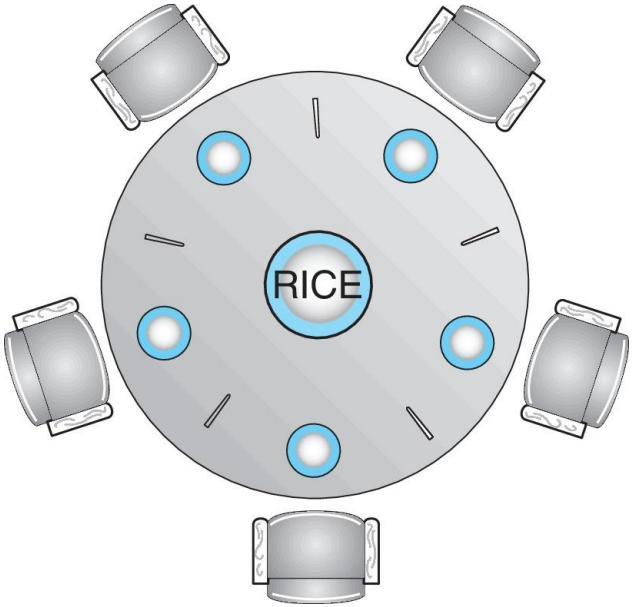
Dining-Philosophers Problem

- Hayatlarını düşünerek ve yemek yiyerek geçiren beş filozof var. Masanın ortasında bir kase pirinç bulunur ve masa beş adet tek çubukla döşenir.
- Zaman zaman bir filozof acıkır ve kendisine en yakın olan iki yemek çubuğunu almaya çalışır. Bir filozof bir seferde yalnızca bir yemek çubuğu alabilir.
  - Filozofun yemek yemek için iki çubuğa ihtiyacı vardır, yedikten sonra ikisini de bırakır.



Dining-Philosophers Problem

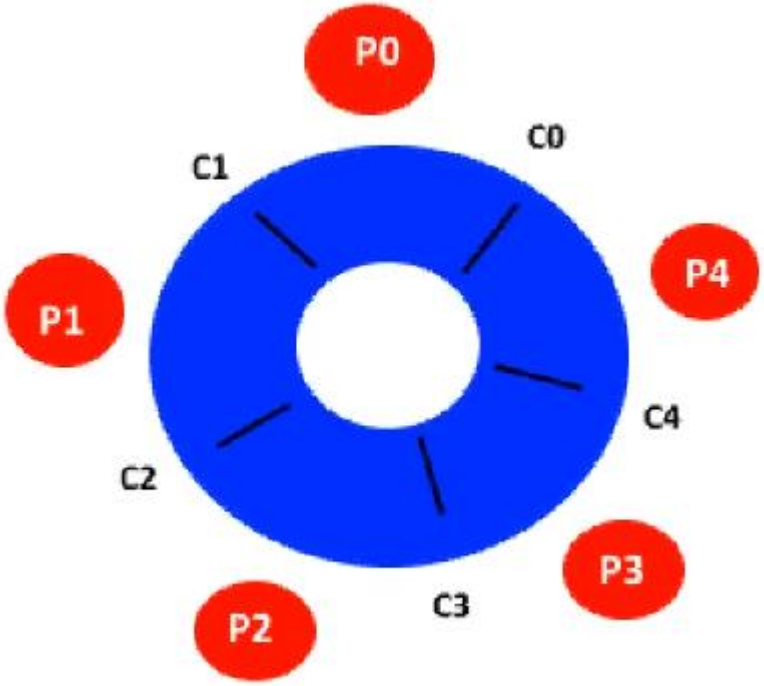
- Herkese yetecek çubuk olmadığından filozofların hepsi aynı anda yemek yiyemez.
- Problem->Filozoflar açlıktan ölmeden yemek yiyebilirler mi, yerlerse nasıl?
- Bu problem, çeşitli kaynakların çeşitli prosesler arasında deadlock ve starvation olmadan bir şekilde tahsis edilmesi ihtiyacının basit bir temsilidir.



Dining-Philosophers Problem

- ***i*** filozofu için Dining-Philosophers probleminin semafor ile çözünüz. İpucu: semafor değişkenini masadaki 5 tane çubuk için 5 elemanlı bir chopstick[] bir dizisi tanımlayınız. ***i*** filozofu için **semaphore chopstick[5];** kullanılacak. **chopstick[]** semafor dizisinin başlangıç değeri ne olmalı?

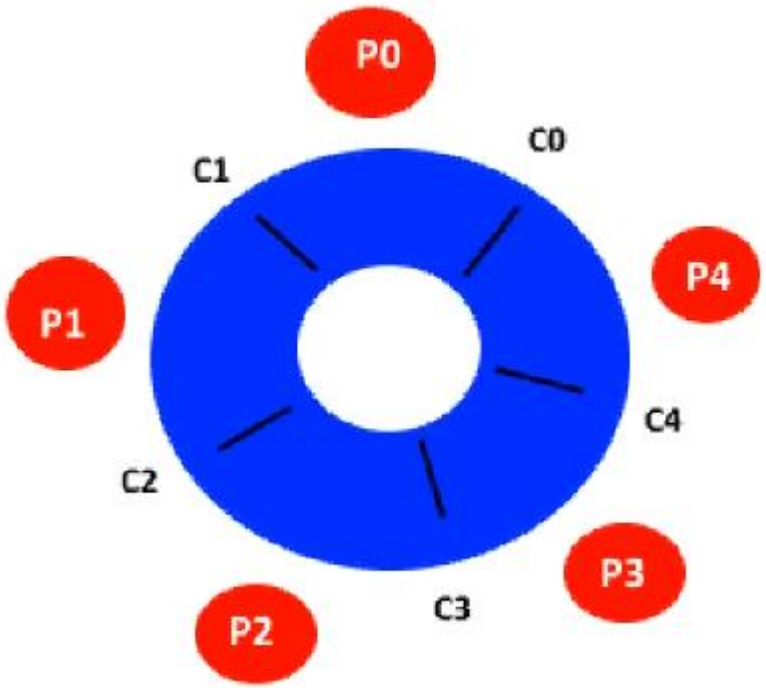
```
while (true){

      ?

      /* eat for awhile */

      ?

      /* think for awhile */

}
```



Dining-Philosophers Problem

- ***i*** filozofu için Dining-Philosophers probleminin semafor ile çözümü:

```
while (true){
    wait (chopstick[i] );
    wait (chopStick[ (i + 1) % 5] );

     /* eat for awhile */

    signal (chopstick[i] );
    signal (chopstick[ (i + 1) % 5] )

     /* think for awhile */

}
```

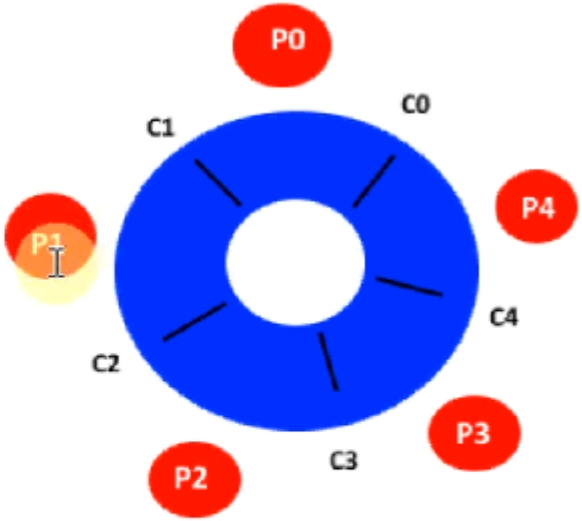


Example-Dining-Philosophers Problem Solution

- Örnek çalıştırma

```
semaphore chopchick[5];

do
{
  think...
wait(chopstick[i]);
wait(chopstick[(i+1)mod 5]);
 ....
 eat
 ....
signal(chopstick[i]);
signal(chopstick[(i+1)mod 5]);
...
think
}while(1);
```

Pi	cp[i]	cp[i+1 mod 5]



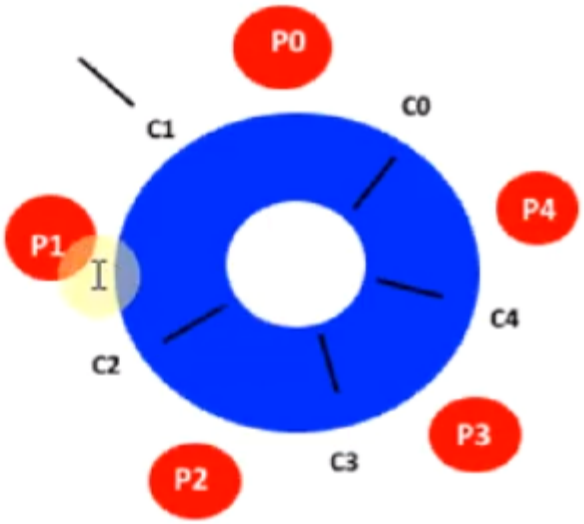
chopstick[i]	0	1	2	3	4
	1	1	1	1	1

Example-Dining-Philosophers Problem Solution

P1 acıkmış olsun wait (chopstick[1]) i çalıştırır ve chopstick[1] 1 azaltılır 0 olur.

```
semaphore chopchick[5];

do
{
  think...
  wait(chopstick[i]);
  wait(chopstick[(i+1)mod 5]);
  ....
  eat
  ....
  signal(chopstick[i]);
  signal(chopstick[(i+1)mod 5]);
  ...
  think
}while(1);
```

Pi	cp[i]	cp[i+1 mod 5]



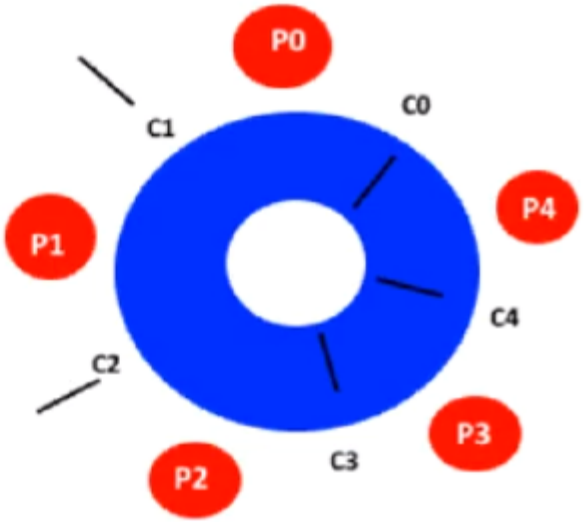
chopstick[i]	0	1	2	3	4
	1	0	1	1	1

Example-Dining-Philosophers Problem Solution

- Ardından wait (chopStick[2]) yi çalıştırır, chopstick[2] 1 azaltılır 0 olur. Bu noktada sağ tarafındaki C2 çubuğunu almıştır.

```
semaphore chopchick[5];

do
{
  think...
wait(chopstick[i]);
wait(chopstick[(i+1)mod 5]);
 ....
 eat
 ....
signal(chopstick[i]);
signal(chopstick[(i+1)mod 5]);
 ...
think
}while(1);
```

Pi	cp[i]	cp[i+1 mod 5]



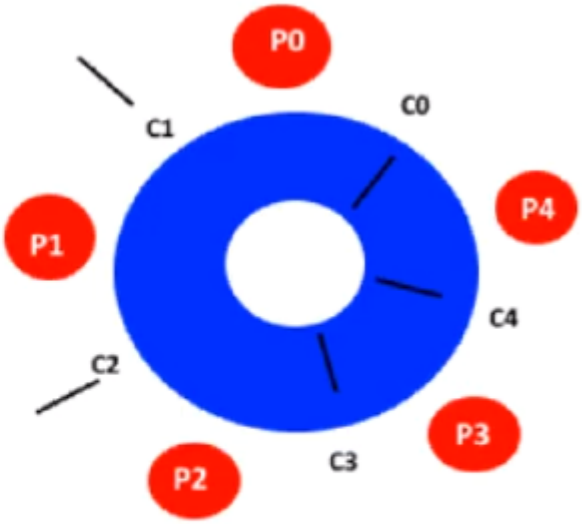
chopstick[i]	0	1	2	3	4
	1	0	0	1	1

Example-Dining-Philosophers Problem Solution

- iki çubuğu alan P1 yemeye başlar.

```
semaphore chopchick[5];

do
{
  think...
wait(chopstick[i]);
wait(chopstick[(i+1)mod 5]);
....
eat        P1
....
signal(chopstick[i]);
signal(chopstick[(i+1)mod 5]);
...
think
}while(1);
```

Pi	cp[i]	cp[i+1 mod 5]

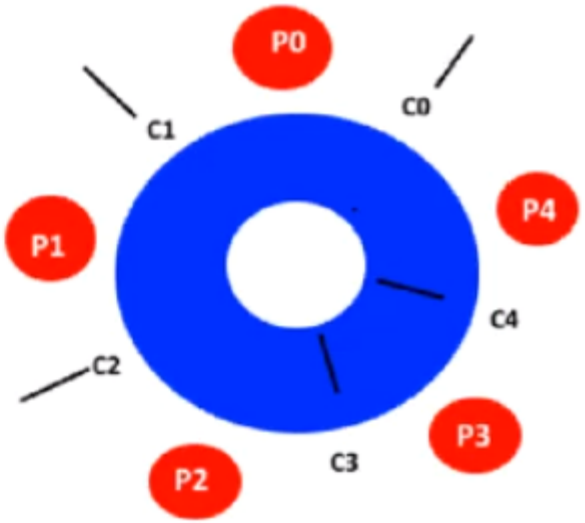


chopstick[i]	0	1	2	3	4
	1	0	0	1	1

Example-Dining-Philosophers Problem Solution

- Bu esnada P1'in komşusu P0 da acıksın, P0 wait(chopstick[0]) çalıştırır sol tarafındaki çubuğu alır , chopstick[0] 1 azaltılır 0 olur

```
semaphore chopchick[5];

do
{

wait(chopstick[i]);          P0
wait(chopstick[(i+1)mod 5]);
....
eat                P1
....
signal(chopstick[i]);
signal(chopstick[(i+1)mod 5]);
...
think
}while(1);
```

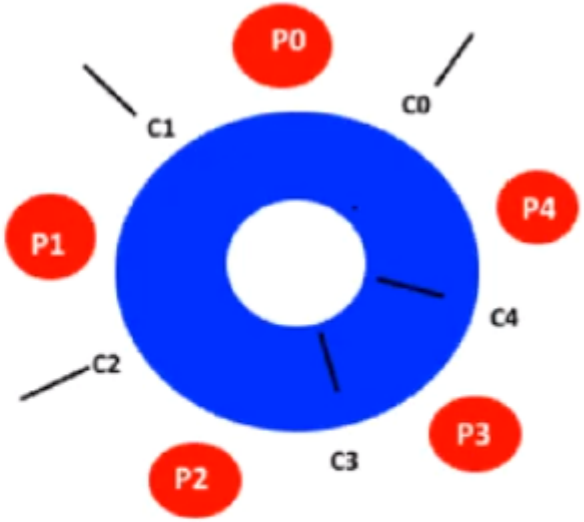


chopstick[i]	0	1	2	3	4
	0	0	0	1	1

Example-Dining-Philosophers Problem Solution

- Ardından P0, wait (chopStick[1]) i çalıştırır ancak chopStick[1] zaten 0 dır, yani başka proses CS'ye giremez

```
semaphore chopchick[5];

do
{

wait(chopstick[i]);                P0
wait(chopstick[(i+1)mod 5]);
....
eat                P1
....
signal(chopstick[i]);
signal(chopstick[(i+1)mod 5]);
...
think
}while(1);
```



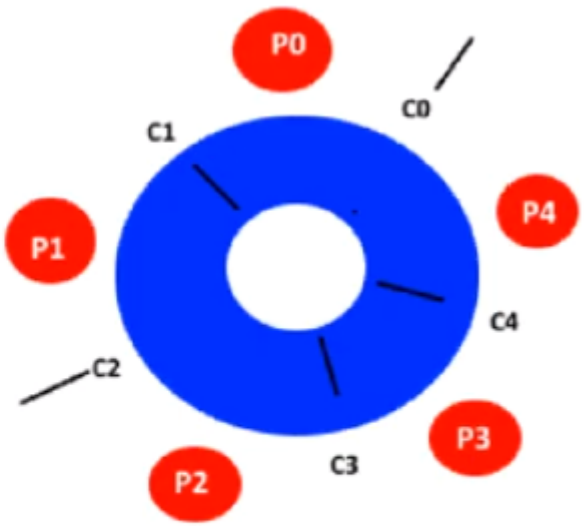
chopstick[i]	0	1	2	3	4
	0	0	0	1	1

Example-Dining-Philosophers Problem Solution

- P1 yemeyi bitirince sol tarafındaki çubuk için signal i çalıştırır, c1 değeri bir artırılır, C1 çubuğu masaya geri koyulmuştur.

```
semaphore chopchick[5];

do
{

wait(chopstick[i]);
wait(chopstick[(i+1)mod 5]);
....
eat
....
signal(chopstick[i]);
signal(chopstick[(i+1)mod 5]);
...
think
}while(1);
```

P0



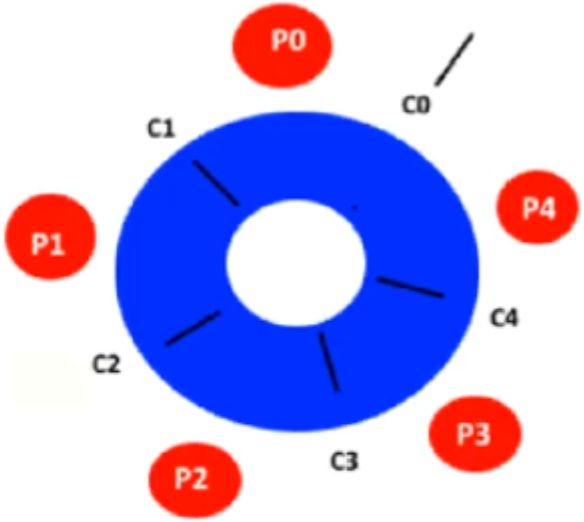
chopstick[i]	0	1	2	3	4
	0	1	0	1	1

Example-Dining-Philosophers Problem Solution

- Daha sonra sağ tarafındaki çubuk için signal i çalıştırır, c2 bir artılılır, c2 çubuğunu masaya geri koymuştur.

```
semaphore chopchick[5];

do
{

wait(chopstick[i]);
wait(chopstick[(i+1)mod 5]);
....
eat
....
signal(chopstick[i]);
signal(chopstick[(i+1)mod 5]);
...
think
}while(1);
```

P0



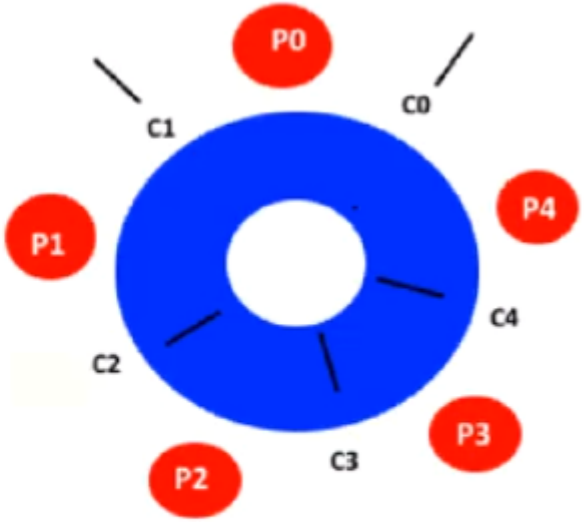
chopstick[i]	0	1	2	3	4
	0	1	1	1	1

Example-Dining-Philosophers Problem Solution

- Bu noktada sağ tarafındaki çubuğu bekleyen P0 çubuğu alabilir, C1 değeri bir azaltılır 0 olur.

```
semaphore chopchick[5];

do
{

wait(chopstick[i]);
wait(chopstick[(i+1)mod 5]);
....
eat
....
signal(chopstick[i]);
signal(chopstick[(i+1)mod 5]);
...
think
}while(1);
```

P0

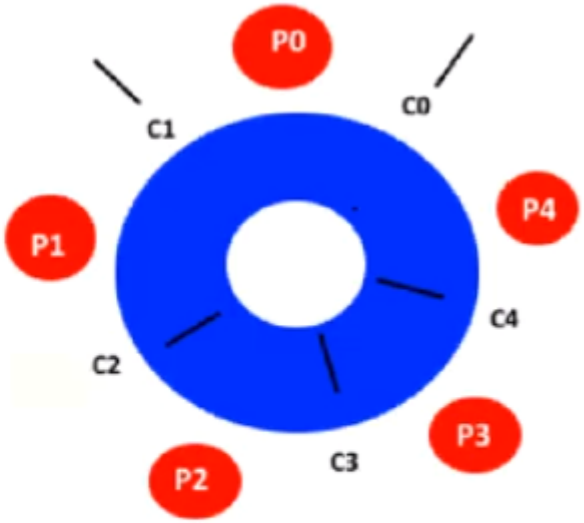


chopstick[i]	0	1	2	3	4
	0	0	1	1	1

Example-Dining-Philosophers Problem Solution

- P0 yemeğini yemeye başlar. Bu çözüme göre iki komşu aynı anda bir çubuğu alamaz, dolayısıyla mutual exclusion sağlanmış olur.

```
semaphore chopchick[5];

do
{

wait(chopstick[i]);
wait(chopstick[(i+1)mod 5]);
....
eat      P0
....
signal(chopstick[i]);
signal(chopstick[(i+1)mod 5]);
...
think
}while(1);
```

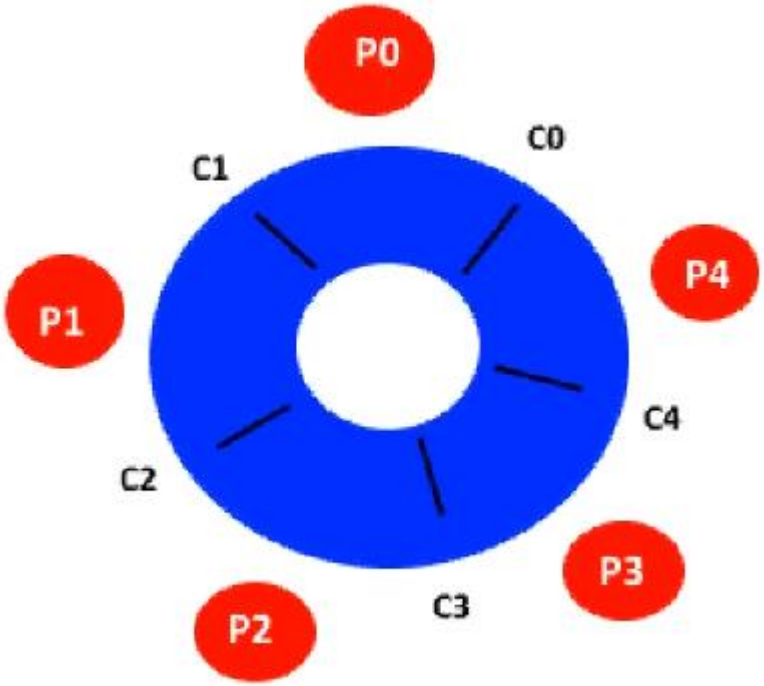


chopstick[i]	0	1	2	3	4
	0	0	1	1	1

Dining-Philosophers Problem

- Beş filozofun da aynı anda acıktığını ve aynı anda **wait(chopstick[i]);** komutunu çalıştırdığını varsayın.
- Her filozof ilk çubuğunu aldıktan hemen sonra ikinci çubuğunu almak için **wait(chopstick[(i+1) % 5]);** komutunu çalıştırıyor.
- Bu durumda sistemin durumu ne olur? Kaç filozof yemek yiyebilir?

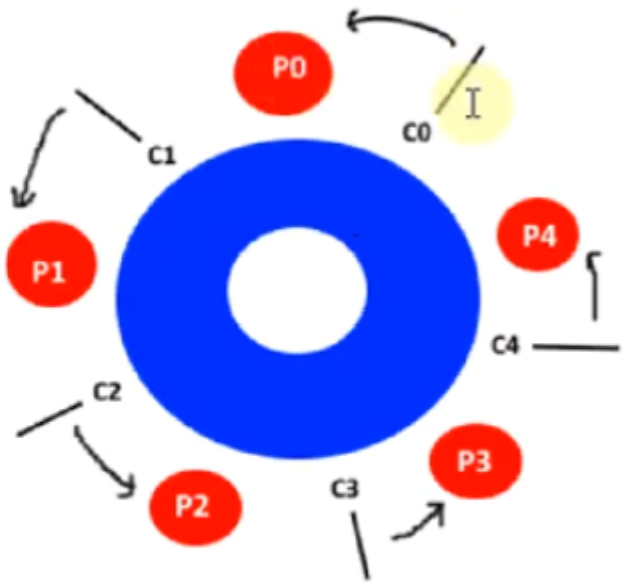
```
while (true){
    wait (chopstick[i] );
    wait (chopStick[ (i + 1) % 5] );

     /* eat for awhile */

    signal (chopstick[i] );
    signal (chopstick[ (i + 1) % 5] );

     /* think for awhile */

}
```



Dining-Philosophers Problem Solution

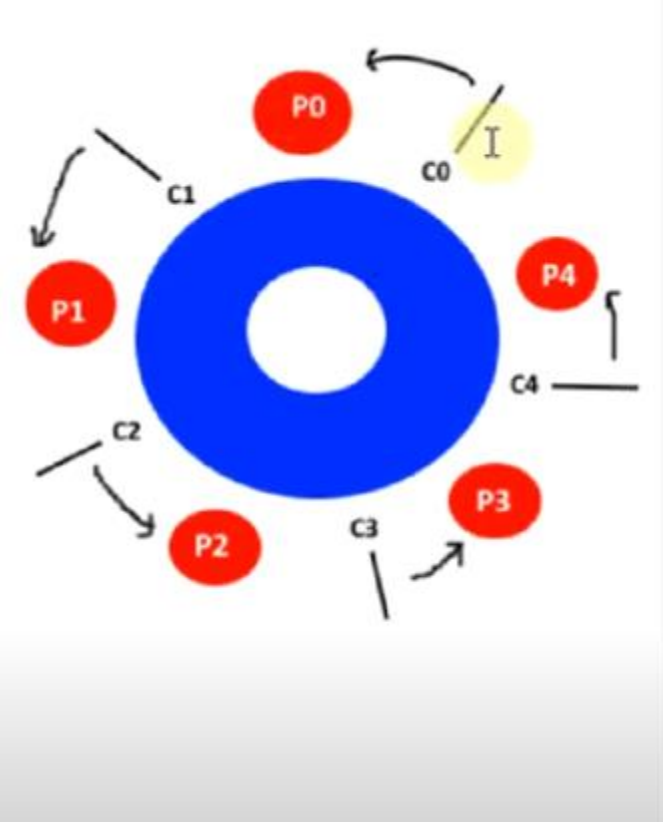
- Bu çözüm, iki komşunun aynı anda yemek yememesini garanti etse de, deadlock problemi yaratabilir:
- Beş filozof da aynı anda acıksın ve her biri sol yemek çubuğunu tutsun. chopstick dizisinin tüm elemanları 0'a eşit olacaktır. Her filozof sağ tarafındaki yemek çubuğunu tutmaya çalıştığında, bu sonsuza kadar ertelenecektir.

```
semaphore chopchick[5];

do
{
 think...
wait(chopstick[i]);
wait(chopstick[(i+1)mod 5]);
 ....
 eat
 ....
signal(chopstick[i]);
signal(chopstick[(i+1)mod 5]);
 ...
 think
 }while(1);
```



Dining-Philosophers Problem Solution

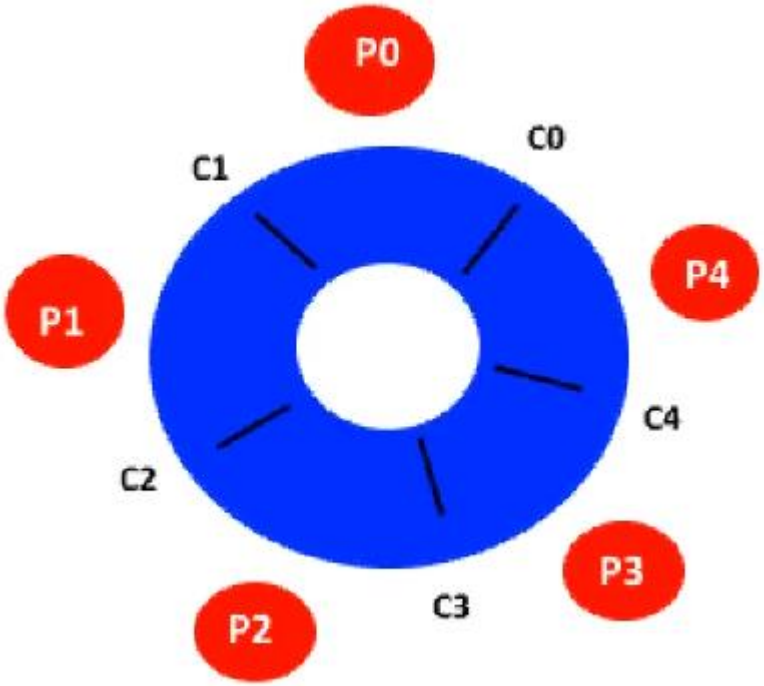
- Deadlock sorununun birkaç olası çözümü şu şekilde olabilir:
  - En fazla dört filozofun aynı anda masada oturmasına izin verilebilir.
  - Bir filozofun yemek çubuklarını yalnızca her iki yemek çubuğu da varsa almasına izin verilebilir (bunu yapmak için, onları kritik bir bölümden alması gerekir).
  - Asimetrik bir çözüm kullanılabilir: tek sayılı bir filozof önce sol sonra da sağ yemek çubuğunu alırken, çift sayılı bir filozof önce sağ sonra da sol yemek çubuğunu alır.



Dining-Philosophers Problem

- Sırayı inceleyin: P0 önce chopstick[0]'ı alıyor ardından chopstick[1]'i de alıyor ve yemeye başlıyor.
- Bu sırada P2 de acıkıyor ve kendi tarafında yemek yemek için ilgili komutlarını yürütmeye başlıyor.
- Bu yürütmede aynı anda kaç filozof yemek yiyebilir?

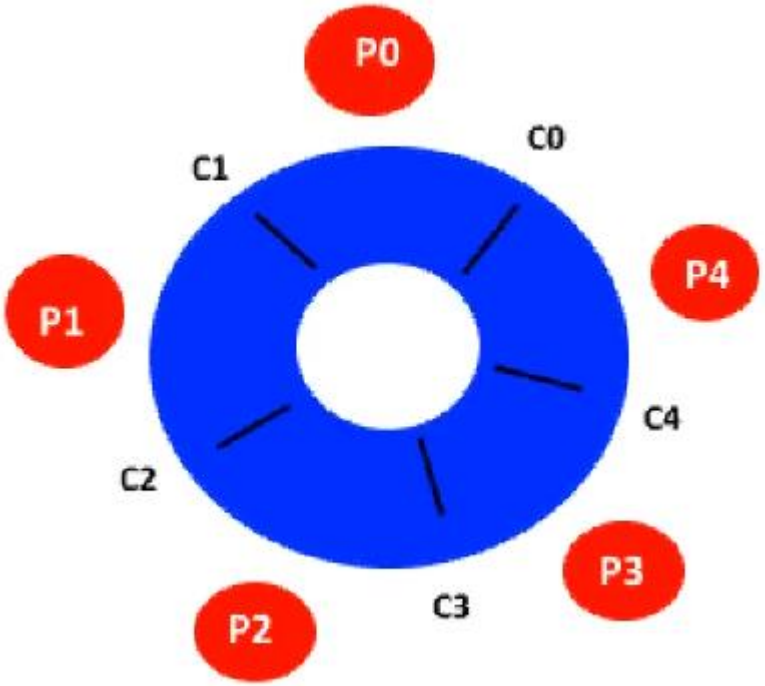
```
while (true){
    wait (chopstick[i] );
    wait (chopStick[ (i + 1) % 5] );

     /* eat for awhile */

    signal (chopstick[i] );
    signal (chopstick[ (i + 1) % 5] );

     /* think for awhile */

}
```



Dining-Philosophers Problem

- Bu semafor çözümünde hangi filozof çiftleri aynı anda yemek yiyebilir, hangileri yiyemez?

```
while (true){
    wait (chopstick[i] );
    wait (chopStick[ (i + 1) % 5] )

     /* eat for awhile */

    signal (chopstick[i] );
    signal (chopstick[ (i + 1) % 5]

     /* think for awhile */

}
```



Process Synchronization

- Processler dersinde shared memory yöntemi kullanılarak verilen aşağıdaki producer-consumer probleminin çözümünü (senkronizasyon sağlanmamıştı) semafor yöntemi kullanarak senkronizasyon çözümü için hangi semafor değişkeni veya değişkenlere ihtiyaç vardır?

```
while (true) {
     /* produce an item in next_produced */

     while (counter == BUFFER_SIZE)
        ; /* do nothing */

     buffer[in] = next_produced;
     in = (in + 1) % BUFFER_SIZE;
     counter++;
}
```

Yeni eleman eklendi.

```
while (true) {
     while (counter == 0)
        ; /* do nothing */

     next_consumed = buffer[out];
     out = (out + 1) % BUFFER_SIZE;
     counter--;

     /* consume the item in next_consumed */
}
```

Bir eleman alındı.

Process Synchronization

- Aşağıdaki semafor değişkenlerinin başlangıç değerleri ne olmalıdır?

semaphore **mutex** ; /* buffer'e erişimde mutuale xclusion'ı sağlamak için kullanılır.*/

semaphore **full**; /* dolu buffer sayısını tutar */

semaphore **empty ;** /* boş buffer sayısını tutar*/

```
while (true) {
     /* produce an item in next_produced */

     while (counter == BUFFER_SIZE)
        ; /* do nothing */

     buffer[in] = next_produced;
     in = (in + 1) % BUFFER_SIZE;
     counter++;
}
```

Yeni eleman eklendi.

```
while (true) {
     while (counter == 0)
        ; /* do nothing */

     next_consumed = buffer[out];
     out = (out + 1) % BUFFER_SIZE;
     counter--;

     /* consume the item in next_consumed */
}
```

Bir eleman alındı.

Bounded-Buffer (also called Producer-Consumer) Problem

- ***n*** elemanlı bir buffer ile producer-consumer gerçekleştirildiğinde buffer'deki eleman sayısını tutan **count** değişkeni için race condition oluşabiliyordu.
- Producer-consumer probleminin semaforla çözümünde prosesler aşağıda başlangıç değerleri verilen veri yapılarını paylaşırlar:
  - int n;
  - semaphore **mutex** = 1; /* buffer'e erişimde mutual exclusion'ı sağlamak için kullanılır.*/
  - semaphore **full** = 0; /* dolu buffer sayısını tutar */
  - semaphore **empty = n;** /* boş buffer sayısını tutar*/

Counting semaphore

Binary semaphore

Bounded Buffer Problem (Cont.)

Producer prosesinin yapısı

```
do {
   ...
   /* producer bir eleman üretiyor*/
   ...
   wait(empty); /* boş buffer yoksa producer ürettiğini
   yerleştirmek için boş buffer oluşmasını bekleyecek */
   wait(mutex); //mutual exclusion için
       /* üretilen eleman buffer'e yerleştiriliyor */
       buffer[in] = next_produced;
       in = (in+1) % BUFFER_SIZE;
       counter++;
       ...
   signal(mutex); /* kullanılan mutex serbest bırakılıyor
   böylece CS'ye girmek isteyen varsa girebilecek */
   signal(full); // dolu buffer sayısı artırılıyor
} while (true);
```

Bounded Buffer Problem (Cont.)

Consumer prosesinin yapısı

```
do {
   wait(full); /* tüketilecek bir eleman buffer'da
   yoksa consumer eleman tüketmek için bekleyecek*/
   wait(mutex); //mutual exclusion için
       /* buffer'den bir eleman çıkarılıyor */
       next_produced = buffer[in];
       out = (out+1) % BUFFER_SIZE;
       counter--;
       ...
   signal(mutex); /* kullanılan mutex serbest bırakılıyor
   böylece CS'ye girmek isteyen girebilecek */
   signal(empty); // boş buffer sayısı artırılıyor
       ...
     /* çıkarılan eleman tüketiliyor*/
     ...
} while (true);
```

Bounded Buffer Problem (Cont.)

- empty ve full semaforları yerine buffer'daki eleman sayısını gösteren count diye bi semafor olsaydı, çözüm yine düzgün çalışırmıydı?

Producer prosesinin yapısı

```
do {
    ...
    /* producer bir eleman üretiyor*/
    ...
    wait(count); /* wait(empty); boş buffer yoksa producer ürettiğini
    yerleştirmek için boş buffer oluşmasını bekleyecek */
    wait(mutex); //mutual exclusion için
        /* üretilen eleman buffer'e yerleştiriliyor */
        buffer[in] = next_produced;
        in = (in+1) % BUFFER_SIZE;
        counter++;
        ...
    signal(mutex); /* kullanılan mutex serbest bırakılıyor böylece
    CS'ye girmek isteyen varsa girebilecek */
    signal(count); // signal(full); dolu buffer sayısı artırılıyor
} while (true);
```

Readers-Writers Problem

- Bir veri seti, bir dizi eşzamanlı proses arasında paylaşılır.
  - **Readers** – bazı prosesler veri seti üzerinde sadece okuma yaparlar, herhangi bir güncelleme yapmazlar
  - **Writers** – bazı prosesler ise veri seti üzerinde güncelleme yaparlar
- İki reader aynı anda paylaşılan verilere erişirse, herhangi bir olumsuz etki ortaya çıkmayacaktır.
- Ancak, bir writer ve başka bir process(bir reader veya bir writer) veritabanına aynı anda erişirse, kaos ortaya çıkabilir.
- **Reader-Writer problemi**
  - **Birden fazla reader'ın aynı anda** okumasına izin verilmesini
  - **Sadece bir writer'ın** paylaşılan verilere **aynı anda erişebilmesinin** nasıl sağlanacağı problemidir.

Readers-Writers Problem Variations

- ***Reader-Writer*** probleminin çeşitli varyasyonları vardır, hepsi de bazı öncelikler içerir.
- ***First*** variation – bir writer paylaşılan veriyi kullanmak için izin almadıkça hiçbir reader'ın bekletilmemesini gerektirir.
  - Başka bir deyişle, bir writer bekliyor diye hiçbir reader diğer reader'lerin bitirmesini beklememelidir.
- ***Second*** variation – bir writer hazır olduğunda, o writer'ın yapacağı güncellemeyi mümkün olan en kısa sürede gerçekleştirmesini gerektirir.
  - Başka bir deyişle, bir writer veriye erişmeyi bekliyorsa, hiçbir yeni reader okumaya başlayamaz.

Readers-Writers Problem Variations

- Her iki varyant için de önerilecek çözümler starvation ile sonuçlanabilir.
  - İlk durumda, writer'ler
  - İkinci durumda, reader'ler açlıktan ölebilir;
- Bu nedenle, problemin başka varyantları önerilmiştir.
- Bazı sistemlerde problem reader-writer locks sağlayan kernel tarafından çözülmüştür.

Readers-Writers Problem

- Burada first variation için semafor kullanılarak çözüm verilmiştir.
- Paylaşılan veriler
  - Data set
  - Semaphore **`rw_mutex`** //initialized to 1
  - Semaphore **`mutex`** //initialized to 1
  - Integer **`read_count`** //initialized to 0
- **`rw_mutex`** writer ve reader processleri için ortaktır, bir writer ve başka bir process(bir reader veya bir writer) arasında mutual exclusion sağlamak için kullanılır.
  - CS'ye giren **ilk** ve CS'den çıkan **son** reader proses tarafından da kullanılır. CS'de daha önceden reader prosesler varsa, bundan sonra CS'ye giren veya çıkan reader proses tarafından kullanılmaz.
- **`Mutex`** semaforu, **`read_count`** değişkeni update edilirken mutual exclusion'ı sağlamak için kullanılır.

Readers-Writers Problem (Cont.)

- Writer proses için semafor çözümünü veriniz. **`rw_mutex`** semaforunun başlangıç değeri ne olmalıdır?

- İpucu: writer proses için sadece **`rw_mutex`** semafor kullanımı yeterlidir. ?

```
while (true) {
      ?
      . . .
   /* writing is performed */
      . . .
      ?
}
```

Readers-Writers Problem (Cont.)

- **writer process**

```
while (true) {
   wait(rw_mutex);
      . . .
   /* writing is performed */
      . . .
   signal(rw_mutex);
}
```

- Aynı anda birden çok writer process CS'de olamaz, bu semaforla sağlanmıştır.

Readers-Writers Problem (Cont.)

- **reader process**

Semaphore **rw_mutex** //initialized to 1
Semaphore **mutex** //initialized to 1
Integer **read_count** //initialized to 0

```
while (true) {
   wait(mutex);
   read_count++;
   if (read_count == 1)
      wait(rw_mutex);
   signal(mutex);
      . . .
   /* reading is performed */
      . . .
   wait(mutex);
   read_count--;
   if (read_count == 0)
      signal(rw_mutex);
   signal(mutex);
}
```

Readers-Writers Problem (Cont.)

- **reader process** çözümünde mutex semaforu ne işe yarar, olmasaydı nasıl bir problem olurdu?

```
while (true) {
   wait(mutex);
   read_count++;
   if (read_count == 1)
      wait(rw_mutex);
   signal(mutex);
      . . .
   /* reading is performed */
      . . .
   wait(mutex);
   read_count--;
   if (read_count == 0)
      signal(rw_mutex);
   signal(mutex);
}
```

Readers-Writers Problem (Cont.)

- **reader process** çözümünde if (read count == 1) kontrolü ne sağlıyor, niçin gereklidir?

```
while (true) {
   wait(mutex);
   read_count++;
   if (read_count == 1)
      wait(rw_mutex);
   signal(mutex);
      . . .
   /* reading is performed */
      . . .
   wait(mutex);
   read_count--;
   if (read_count == 0)
      signal(rw_mutex);
   signal(mutex);
}
```

Readers-Writers Problem (Cont.)

- **reader process** çözümünde if (read count == 1) kontrolü ne sağlıyor, niçin gereklidir?

```
while (true) {
   wait(mutex);
   read_count++;
   if (read_count == 1)
      wait(rw_mutex);
   signal(mutex);
      . . .
   /* reading is performed */
      . . .
   wait(mutex);
   read_count--;
   if (read_count == 0)
      signal(rw_mutex);
   signal(mutex);
}
```

Readers-Writers Problem (Cont.)

- **reader process** çözümünde if (read count == 1) kontrolü olmasaydı nolurdu?

```
while (true) {
   wait(mutex);
   read_count++;
   if (read_count == 1)
      wait(rw_mutex);
   signal(mutex);
      . . .
   /* reading is performed */
      . . .
   wait(mutex);
   read_count--;
   if (read_count == 0)
      signal(rw_mutex);
   signal(mutex);
}
```

Readers-Writers Problem (Cont.)

- **reader process** çözümünde if (read count == 0) kontrolü ne sağlıyor, niçin gereklidir?

```
while (true) {
   wait(mutex);
   read_count++;
   if (read_count == 1)
      wait(rw_mutex);
   signal(mutex);
      . . .
   /* reading is performed */
      . . .
   wait(mutex);
   read_count--;
   if (read_count == 0)
      signal(rw_mutex);
   signal(mutex);
}
```

Readers-Writers Problem (Cont.)

- **reader process** çözümünde if (read count == 0) kontrolü olmasaydı nolurdu?

```
while (true) {
   wait(mutex);
   read_count++;
   if (read_count == 1)
      wait(rw_mutex);
   signal(mutex);
      . . .
   /* reading is performed */
      . . .
   wait(mutex);
   read_count--;
   if (read_count == 0)
      signal(rw_mutex);
   signal(mutex);
}
```

Readers-Writers Problem Variations

- İstenen senkronizasyonun sağlanıp sağlanmadığını kontrol edelim

```
while (true) {
   wait(mutex);
   read_count++;
   if (read_count == 1)
      wait(rw_mutex);
   signal(mutex);
      . . .
   /* reading is performed
      . . .
   wait(mutex);
   read_count--;
   if (read_count == 0)
      signal(rw_mutex);
   signal(mutex);
}
```

read_count = 0

rw_mutex=1

mutex = 1

Shared file

```
while (true) {
   wait(rw_mutex);
      . . .
   /* writing is performed
      . . .
   signal(rw_mutex);
}
```

W	no W
W	no R
R	no W
R	R

Readers-Writers Problem Variations

- Bir **writer** CS'de iken başka **writer** CS'ye girmek istesin

```
while (true) {
   wait(mutex);
   read_count++;
   if (read_count == 1)
      wait(rw_mutex);
   signal(mutex);
      . . .
   /* reading is performed
      . . .
   wait(mutex);
   read_count--;
   if (read_count == 0)
      signal(rw_mutex);
   signal(mutex);
}
```

read_count = 0

rw_mutex=0

mutex = 1

Shared file

```
while (true) {
   wait(rw_mutex);
      . . .
   /* writing is performed
      . . .
   signal(rw_mutex);
}
```

W	no W
W	no R
R	no W
R	R

Readers-Writers Problem Variations

- Bir **writer** CS'de iken bir **reader** CS'ye girmek istedeğinde ne olur?

```
while (true) {
   wait(mutex);
   read_count++;
   if (read_count == 1)
      wait(rw_mutex);
   signal(mutex);
      . . .
   /* reading is performed
      . . .
   wait(mutex);
   read_count--;
   if (read_count == 0)
      signal(rw_mutex);
   signal(mutex);
}
```

read_count = 0

rw_mutex=0

mutex = 0

Shared file

```
while (true) {
   wait(rw_mutex);
      . . .
W1 /* writing is performed
      . . .
   signal(rw_mutex);
}
```

W	no W
W	no R
R	no W
R	R

Readers-Writers Problem Variations

- Bir **writer** CS'de iken bir **reader** CS'ye girmek istesin

```
while (true) {
   wait(mutex);
   read_count++;
   if (read_count == 1)
R1  wait(rw_mutex);
   signal(mutex);
      . . .
   /* reading is performed
      . . .
   wait(mutex);
   read_count--;
   if (read_count == 0)
      signal(rw_mutex);
   signal(mutex);
}
```

read_count = 1

rw_mutex=0

mutex = 0

Shared file

```
while (true) {
   wait(rw_mutex);
      . . .
W1 /* writing is performed
      . . .
   signal(rw_mutex);
}
```

W	no W
W	no R
R	no W
R	R

Readers-Writers Problem Variations

- **writer** hala CS'de iken başka **reader'ler** de CS'ye girmek isteyince ne olur?

```
while (true) {
   wait(mutex);
   read_count++;
   if (read_count == 1)
R1  wait(rw_mutex);
   signal(mutex);
      . . .
   /* reading is performed
      . . .
   wait(mutex);
   read_count--;
   if (read_count == 0)
      signal(rw_mutex);
   signal(mutex);
}
```

read_count = 1

rw_mutex=0

mutex = 0

Shared file

```
while (true) {
   wait(rw_mutex);
      . . .
W1 /* writing is performed
      . . .
   signal(rw_mutex);
}
```

W	no W
W	no R
R	no W
R	R

Readers-Writers Problem Variations

- **writer** hala CS'de iken başka **reader'ler** CS'ye girmek istesin

R2 R3 R4

```
while (true) {
   wait(mutex);
   read_count++;
   if (read_count == 1)
R1  wait(rw_mutex);
   signal(mutex);
      . . .
   /* reading is performed
      . . .
   wait(mutex);
   read_count--;
   if (read_count == 0)
      signal(rw_mutex);
   signal(mutex);
}
```

read_count = 1

rw_mutex=0

mutex = 0

Shared file

```
while (true) {
   wait(rw_mutex);
      . . .
W1 /* writing is performed
      . . .
   signal(rw_mutex);
}
```

W	no W
W	no R
R	no W
R	R

Readers-Writers Problem Variations

- Bir **reader** CS'de iken bir **writer** CS'ye girmek istediğinde ne olur?

```
while (true) {
   wait(mutex);
   read_count++;
   if (read_count == 1)
      wait(rw_mutex);
   signal(mutex);
R1 . . .
   /* reading is performed
      . . .
   wait(mutex);
   read_count--;
   if (read_count == 0)
      signal(rw_mutex);
   signal(mutex);
}
```

read_count = 1

rw_mutex=0

mutex = 1

Shared file

```
while (true) {
   wait(rw_mutex);
      . . .
   /* writing is performed
      . . .
   signal(rw_mutex);
}
```

W	no W
W	no R
R	no W
R	R

Readers-Writers Problem Variations

- Bir **reader** CS'de iken bir **writer** CS'ye girmek istesin

```
while (true) {
   wait(mutex);
   read_count++;
   if (read_count == 1)
      wait(rw_mutex);
   signal(mutex);
R1 . . .
   /* reading is performed
      . . .
   wait(mutex);
   read_count--;
   if (read_count == 0)
      signal(rw_mutex);
   signal(mutex);
}
```

read_count = 1

rw_mutex=0

mutex = 1

Shared file

```
while (true) {
W1 wait(rw_mutex);
      . . .
   /* writing is performed
      . . .
   signal(rw_mutex);
}
```

W	no W
W	no R
R	no W
R	R

Readers-Writers Problem Variations

- Bir **reader** CS'de iken başka bir **readerler** CS'ye girmek istediğinde ne olur?

```
while (true) {
   wait(mutex);
   read_count++;
   if (read_count == 1)
      wait(rw_mutex);
   signal(mutex);
R1 . . .
   /* reading is performed
      . . .
   wait(mutex);
   read_count--;
   if (read_count == 0)
      signal(rw_mutex);
   signal(mutex);
}
```

read_count = 1

rw_mutex=0

mutex = 1

Shared file

```
while (true) {
   wait(rw_mutex);
      . . .
   /* writing is performed
      . . .
   signal(rw_mutex);
}
```

W	no W
W	no R
R	no W
R	R

Readers-Writers Problem Variations

- devam

```
while (true) {
   wait(mutex);
   read_count++;
   if (read_count == 1)
      wait(rw_mutex);
   signal(mutex);
   R1   R2
   /* reading is performed
      . . .
   wait(mutex);
   read_count--;
   if (read_count == 0)
      signal(rw_mutex);
   signal(mutex);
}
```

read_count = 2

rw_mutex=0

mutex = 1

Shared file

```
while (true) {
   wait(rw_mutex);
      . . .
   /* writing is performed
      . . .
   signal(rw_mutex);
}
```

W	no W
W	no R
R	no W
R	R

Readers-Writers Problem Variations

- Bir başka **reader** daha CS'ye girmek istesin

```
while (true) {
   wait(mutex);
   read_count++;
   if (read_count == 1)
      wait(rw_mutex);
   signal(mutex);
   R1   R2   R3
   /* reading is performed
      . . .
   wait(mutex);
   read_count--;
   if (read_count == 0)
      signal(rw_mutex);
   signal(mutex);
}
```

read_count = 3

rw_mutex=0

mutex = 1

Shared file

```
while (true) {
   wait(rw_mutex);
      . . .
   /* writing is performed
      . . .
   signal(rw_mutex);
}
```

W	no W
W	no R
R	no W
R	R